`tests/dynamic/suites/benchmark_suite.sh` — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Authored for P10. Honest-SKIPs (`topology_unsupported`, §11.4.69) until the live `dynamic` stack lands; parse-clean + authored today.

> Companion (§11.4.18) to the in-source documentation block at the top of the script.

Overview

§11.4.169 **performance / benchmark** suite. It measures per-request latency through the live `dynamic` stack, computes **p50/p95/p99** from real captured per-request times, and asserts (a) p95 is within an absolute **budget** and (b) — when a recorded baseline exists — there is **no regression** beyond a tolerance vs that baseline. It records the full latency series + percentiles as the captured evidence (§11.4.69); a regression vs baseline is a finding (§11.4.169 benchmarking clause).

Prerequisites

- Source library `tests/dynamic/lib/analyzer_common.sh` + committed `tests/lib/evidence.sh`.
- `curl`, `awk`, `sort`, POSIX `sh`, `date`.
- For a real run: `HELIX_DYNAMIC_STACK=1` + `HELIX_PROXY_URL`.

Usage examples

```bash
# Today (no live stack): honest SKIP, exit 0:
bash tests/dynamic/suites/benchmark_suite.sh

# P10 live run:
HELIX_DYNAMIC_STACK=1 HELIX_PROXY_URL=http://127.0.0.1:34128 \
  BENCH_N=200 BENCH_P95_BUDGET_MS=800 BENCH_REGRESS_PCT=25 \
  bash tests/dynamic/suites/benchmark_suite.sh
```

Env: `BENCH_N` (samples, default 200), `BENCH_TARGET` (default `http://target-a.internal/`), `BENCH_P95_BUDGET_MS` (default 800), `BENCH_REGRESS_PCT` (max allowed p95 growth vs baseline, default 25), `BENCH_BASELINE` (default `qa-results/p9-harness/bench_baseline.p95`).

Edge cases

- **Live stack absent** → `topology_unsupported` SKIP, exit `0`.
- **No baseline file yet** → budget-only check; a passing run **writes** the baseline so future runs gain the regression check.
- **p95 over budget** OR **regression > `BENCH_REGRESS_PCT`** vs the baseline → `FAIL` citing `benchmark.evidence`.

§11.4.115 RED_MODE polarity

`RED_MODE`	What it asserts	PASS means
`0` (default)	GREEN guard	p95 ≤ budget AND no regression beyond tolerance
`1`	RED baseline	the pre-fix stack's **p95 exceeds budget** (regression) → defect reproduced

A `RED_MODE=1` run where p95 stayed within budget is a finding (`no regression to reproduce`), not a pass.

Internal behaviour

- `#!/usr/bin/env bash`, POSIX-clean (`sh -n` + `bash -n`, §11.4.67).
- Captures `BENCH_N` `%{time_total}` samples (converted to integer ms via `awk`) into `latency_ms.series`; nearest-rank `awk` computes p50/p95/p99.
- Reads the baseline p95 (if any), computes the regression %, writes `benchmark.evidence` (percentiles + series). GREEN requires within-budget AND no-regression; on pass it refreshes the baseline file. Evidence dir: `qa-results/p9-harness/benchmark_<run-id>/`.
- Resources: shell+curl only; bounded sample count; §12.6 60% host ceiling.

Related

- `tests/dynamic/lib/analyzer_common.sh` — sourced base.

- `tests/dynamic/suites/run_all_suites.sh` — orchestrates this suite.
- Constitution §11.4.169 / §11.4.24 / §11.4.50 / §11.4.69 / §11.4.115 / §12.6; design spec §13.

Last verified

2026-07-01 — run with no live stack: honest SKIP, exit `0`; `sh -n` + `bash -n` parse-clean. Live latency measurement + baseline regression check are exercised in **P10**.